SSW 590 — DevOps

Assignment 4: Cloud Docker Deployment and Project Proposal

Version 2.1

Group 1 Submission

by

Group 1: Brahmi Dalvi and Sakshi Deshpande

Stevens Institute of Technology

October 6, 2026

© Stevens Institute of Technology 2026
ALL RIGHTS RESERVED

Template Attribution

This DevOps assignment template was developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**.

SSW 590 — DevOps

Assignment 4: Cloud Docker Deployment and Project Proposal

Version 2.1
Group 1 Submission

Group 1: Brahmi Dalvi and Sakshi Deshpande
Stevens Institute of Technology
October 6, 2026

This group document records four SSW 590 activities completed by Group 1. We deploy a Dockerized two-button website to a cloud virtual machine on Amazon Web Services (AWS), which the group chose in place of a Digital Ocean Droplet; refactor the website's button logic into class-based JavaScript and document it with a UML class diagram; build a Docker container that compiles LaTeX with TeX Live, the same distribution Overleaf uses; and propose the semester project, a side-by-side DevSecOps comparison of two CI/CD tools delivering the same containerized application to AWS. All code, deployment scripts, and evidence live in the group repository so that every result can be reproduced.

Document History

How to Use This Chapter

Add the newest approved change at the top. Retain every approved entry and link each entry to the changed chapter, table, figure, or controlled artifact. Author initials: BD = Brahmi Dalvi, SD = Sakshi Deshpande, CC = Claude Code (AI assistant; see the AI Prompt Log), DDM = Darian Muresan, CDX = OpenAI Codex.

Table 1: Document Update History

Date / Version	Author Initials	Change	Changed At
October 7, 2026 / 2.1	BD, CC	Executed the AWS EC2 deployment described in the AWS Docker Deployment chapter: launched instance `i-00ee7edd4a5352f05` (`t3.micro`, Amazon Linux 2023) with the `launch-ec2.sh` key pair and security group. The bootstrap script's anonymous `git clone` failed because the group repository is private; diagnosed from the real EC2 console output and fixed by adding a read-only GitHub deploy key and completing the clone, Docker build, and container run by hand over SSH. A browser could not open the resulting plain-HTTP site (silent HTTPS upgrade on `amazonaws.com` hosts hitting a closed port 443), diagnosed with `curl`/`Invoke-WebRequest`/`Test-NetConnection` from a working machine against a failing browser and phone; fixed by opening port 443 and adding an `nginx` TLS reverse proxy with a self-signed certificate, which itself needed a stray default `listen 80` server block removed from `nginx.conf`. Verified the live site over both HTTP and HTTPS, captured a real browser screenshot, and marked every row of the deployment checklist **PASS**. Set the real `\WebsiteURL` and updated the Security Notes section to match.	Chapter 2

Document Update History (continued)

Date / Version	**Author Initials**	**Change**	**Changed At**
October 6, 2026 / 2.0	SD, CC	Created the Group 1 document in the group repository from the individual template. Added the AWS Docker Deployment chapter (AWS EC2 in place of a Digital Ocean Droplet, with Console and CLI steps in `minted`), the Class-Based Two-Button Website chapter with a UML class diagram and unit tests, the LaTeX Docker chapter (TeX Live container with real build output), and the Project Proposal chapter (TwinTrack: GitHub Actions vs. Jenkins, dashboard, modified SWOT charts). Switched the build to the LaTeX container and added AWS, EC2, UML, DORA, and SWOT glossary terms.	Chapter 2, Chapter 3, Chapter 4, Chapter 5, Chapter A
September 2, 2026 / 1.0–1.3	DDM, CDX	Created the reusable SSW 590 DevOps assignment template, the one-file-per-chapter convention, and the authoring guide.	Chapter A, Chapter B
[date / version]	*[initials]*	*[approved change and rationale]*	*[link]*

Table of Contents

Template Attribution iii

Document History v

1 Introduction 1
- 1.1 Purpose . . . 1
- 1.2 Team and Repository . . . 1
- 1.3 Repository Layout . . . 1
- 1.4 Chapter Map . . . 2
- 1.5 Use of AI . . . 2

2 AWS Docker Deployment 3
- 2.1 Overview . . . 3
- 2.2 Digital Ocean to AWS Mapping . . . 3
- 2.3 Prerequisites and Account Security . . . 4
- 2.4 Deployment Steps . . . 4
  - 2.4.1 The Bootstrap Script . . . 5
  - 2.4.2 Private Repository and the Deploy Key . . . 5
  - 2.4.3 Path A: AWS Console . . . 5
  - 2.4.4 Path B: AWS CLI . . . 6
  - 2.4.5 Manual Commands on the Instance . . . 7
- 2.5 Verification . . . 8
  - 2.5.1 Pre-deployment Check on a Workstation . . . 8
  - 2.5.2 Cloud Verification . . . 8
- 2.6 Updating the Deployment . . . 10
- 2.7 Security Notes . . . 11
- 2.8 Cost and Teardown . . . 11

3 Class-Based Two-Button Website 12
- 3.1 Overview . . . 12
- 3.2 Original Design . . . 12
- 3.3 Class-Based Design . . . 12
  - 3.3.1 UML Class Diagram . . . 13

3.4 Implementation . . . 13
3.5 Testing . . . 16

4 LaTeX Docker **18**
4.1 Overview . . . 18
4.2 How It Works . . . 18
4.3 Project Files . . . 18
4.4 Commands Run . . . 20
4.4.1 Build the Image . . . 20
4.4.2 Compile the Document . . . 20
4.4.3 Compile This Group Document . . . 21
4.5 Comparison with Overleaf . . . 22
4.6 Troubleshooting . . . 22

5 Project Proposal **23**
5.1 Title . . . 23
5.2 Project Description . . . 23
5.2.1 The Application . . . 23
5.2.2 Sample Tasks . . . 24
5.3 DevSecOps Tools . . . 24
5.4 Pipeline Design . . . 26
5.5 Environment . . . 26
5.6 Operations Dashboard . . . 26
5.7 Comparison Criteria . . . 27
5.8 Preliminary Modified SWOT Charts . . . 27
5.9 What Makes This Project Unique . . . 27
5.10 Mapping to the Project Rubric . . . 29
5.11 Team Roles and Timeline . . . 30

Bibliography **31**

A Document Authoring Guide **32**
A.1 How to Use This Appendix . . . 32
A.2 Template Provenance . . . 32
A.3 File Naming and Responsibilities . . . 32
A.4 Synonyms and Controlled Terms . . . 32
A.5 Building the PDF . . . 33
A.5.1 Group Repository Build (Docker) . . . 33
A.5.2 Local Toolchain Build . . . 33
A.6 Install LaTeX Tools on macOS . . . 33
A.6.1 Recommended Installation . . . 33
A.6.2 Verification and Compilation . . . 34
A.7 Install LaTeX Tools on Windows . . . 34
A.7.1 Recommended Installation . . . 34
A.7.2 Verification and Compilation . . . 34

A.8 Common Build Problems . . . 35
A.9 Authoring and Review Rules . . . 35

B AI Agent Usage Guide **36**
B.1 How to Use This Appendix . . . 36
B.2 Governing Principles . . . 36
B.3 Chapter File Convention . . . 36
B.4 AI Quick Start . . . 37
B.5 Task Routing . . . 37
B.6 DevOps-Specific Rules . . . 37
B.7 Example AI Requests . . . 38
B.7.1 Create an Assignment Brief . . . 38
B.7.2 Review a Student Workflow . . . 38
B.8 Human Decisions and Information Boundaries . . . 38
B.9 Completion Checklist . . . 38

C AI Prompt Log **40**
C.1 How to Use This Appendix . . . 40
C.2 Record: Group Document, AWS Deployment, LaTeX Docker, and Proposal . . . 40
C.3 Record: AWS Deployment Execution and Report Fixes . . . 41

Glossary **42**

Acronyms **43**

Index **44**

List of Tables

1 Document Update History vi

2.1 Digital Ocean Concepts and Their AWS Equivalents 4
2.2 Deployment Checklist 10

4.1 Overleaf Compared with the LaTeX Container 22

5.1 Sample Project Tasks 24
5.2 Planned DevSecOps Tool Chain 24
5.3 How the Two CI/CD Tools Will Be Compared 27
5.4 Preliminary Modified SWOT: GitHub Actions 28
5.5 Preliminary Modified SWOT: Jenkins 28
5.6 Preliminary Modified SWOT: Prometheus and Grafana 28
5.7 Preliminary Modified SWOT: Amazon CloudWatch 29
5.8 Proposal Coverage of the Project Expectations 29

B.1 AI Task-Routing Table 37

List of Figures

2.1 The deployed site at the public EC2 address, on load. 9
2.2 The same deployment after clicking *Blue*. 10
2.3 The same deployment after clicking *Red*. Together with Figures 2.1 and 2.2, this confirms the class-based code from Chapter 3 runs correctly on the real cloud deployment, not just locally. 10

3.1 UML class diagram of the class-based two-button website. 13

4.1 Top of `hello.pdf` compiled by the `latex-builder` container. 21

Chapter 1

Introduction

1.1 Purpose

This is the shared Group 1 document for SSW 590. It collects the group's deployment work, supporting containers, and project proposal in one place, and it is maintained in the group repository alongside the code it describes.

1.2 Team and Repository

Item	Value
Group	Group 1
Members	Brahmi Dalvi (BD), Sakshi Deshpande (SD)
Repository	https://github.com/brahmi9811/ssw-590-group1
Cloud provider	AWS (disclosed group substitution for the assignment's Digital Ocean Droplet; see Chapter 2)
Due date	October 6, 2026

1.3 Repository Layout

```
ssw-590-group1/
  website/color-buttons-app/   two-button site (class-based JS) + Dockerfile
  deploy/aws/                  EC2 launch, user-data, redeploy, teardown scripts
  latex-docker/                TeX Live container that compiles LaTeX
  evidence/                    captured command output from real runs
  docs/                        UML source (Mermaid) and other notes
  report/                      this LaTeX document
```

1.4 Chapter Map

Assignment item	Where it is answered
1–2. Cloud Docker deployment	Chapter 2
3. Class-based JavaScript and UML	Chapter 3
4. LaTeX Docker	Chapter 4
5–6. Project proposal and rubric	Chapter 5
7. Document history	Document History (front matter)

1.5 Use of AI

AI assistance is recorded in Chapter C. Members reviewed every command and result before including it.

Chapter 2

AWS Docker Deployment

2.1 Overview

Deviation from the assignment, disclosed

This assignment's instructions ask for a **Digital Ocean Droplet** running Docker. Group 1 deployed to an **AWS EC2 instance** instead, performing the exact same role: one cloud virtual machine running Docker and serving the website. This substitution is a group choice, not something the assignment itself offered for this item; it is made because the semester project (Chapter 5) is built on AWS, and standing up the account, IAM users, budget alert, and security-group model here means that work is reused rather than duplicated on a second cloud provider the group would not otherwise use. Table 2.1 maps every Digital Ocean concept the assignment describes onto its AWS equivalent, so the deployment can be read and graded as a direct substitute. If a literal Digital Ocean Droplet is required, the group will redeploy there using `doctl` in place of the AWS CLI; the Docker image and application are unchanged either way.

The cloud server is an Amazon Elastic Compute Cloud (EC2) instance, which plays the role of a Droplet. It runs the two-button website from Chapter 3 in a Docker container and serves it on port 80.

Live website

Website: `https://ec2-54-80-252-134.compute-1.amazonaws.com`
Health check: `https://ec2-54-80-252-134.compute-1.amazonaws.com/health`
Source: https://github.com/brahmi9811/ssw-590-group1

The address is set by the `\WebsiteURL` macro in `devopsAssignment.tex`. Replace the placeholder with the public DNS name printed by `launch-ec2.sh` (Section 2.4.4).

2.2 Digital Ocean to AWS Mapping

Table 2.1: Digital Ocean Concepts and Their AWS Equivalents

Digital Ocean	AWS	Used here
Droplet	EC2 instance	`t3.micro`, Free Tier eligible
Droplet image (Ubuntu)	AMI	Amazon Linux 2023
Cloud Firewall	security group	HTTP 80 and HTTPS 443 open; SSH 22 from one IP
SSH key	EC2 key pair	`ssw590-group1-key`
Cloud-init user data	EC2 user data	`deploy/aws/ec2-user-data.sh`
`doctl` CLI	AWS CLI v2	`deploy/aws/launch-ec2.sh`
Docker 1-Click Droplet	Docker installed by `dnf`	Docker from the AL2023 repository

2.3 Prerequisites and Account Security

1. Create an AWS account and enable multi-factor authentication on the root user. Do not use the root user for daily work.
2. In IAM, create a user for each member with console access and the `AmazonEC2FullAccess` policy (or a narrower custom policy), and create an access key for CLI use. Keys are stored only in `~/.aws/credentials` on the member's machine and are never committed.
3. In Billing, create a monthly budget (for example $5) with an e-mail alert so a forgotten instance is noticed quickly.
4. Install and configure the AWS CLI v2:

```
brew install awscli                 # macOS; see AWS docs for Windows/Linux
aws configure                       # access key, secret, region us-east-1, output json
aws sts get-caller-identity         # confirms the CLI is signed in as the IAM user
```

2.4 Deployment Steps

Two equivalent paths are documented: the AWS Console (closest to creating a Droplet in the Digital Ocean web UI) and a scripted CLI path that the group uses for repeatable deployments. Both run the same user data script.

2.4.1 The Bootstrap Script

At first boot the instance runs this script as root. It installs Docker and Git, clones the group repository, builds the image, and starts the container with a restart policy so the site comes back after a reboot. Port 80 on the host is mapped to port 3000 inside the container.

```
#!/bin/bash
# EC2 user data for Amazon Linux 2023: installs Docker, clones the group
# repository, builds the two-button website image, and serves it on port 80.
set -euxo pipefail

dnf update -y
dnf install -y docker git
systemctl enable --now docker
usermod -aG docker ec2-user

git clone https://github.com/brahmi9811/ssw-590-group1.git /opt/ssw-590-group1
cd /opt/ssw-590-group1/website/color-buttons-app

docker build -t color-buttons-app:2.0 .
docker run -d --name color-buttons-app --restart unless-stopped -p 80:3000
↪  color-buttons-app:2.0
```

2.4.2 Private Repository and the Deploy Key

What actually happened on this deployment

The group repository is **private**. The bootstrap script in Section 2.4.1 clones it anonymously over HTTPS, which works only for a public repository. On the real run, cloud-init's user data step failed with `fatal: could not read Username for 'https://github.com': No such device or address`, so the container never started automatically. This was diagnosed from the instance's console output (`aws ec2 get-console-output`) rather than guessed.

The fix was to generate an `ed25519` key pair on the instance, add its *public* half as a read-only **deploy key** on the GitHub repository (Settings → Deploy keys), and then run the same clone, build, and run commands shown in Section 2.4.5 by hand over SSH, using `git@github.com:...` instead of the HTTPS URL. The private key never leaves the instance; the deploy key is read-only and scoped to this one repository. A future improvement would be to bake the deploy key into the AMI or pass it through user data (e.g. via AWS Secrets Manager) so first boot succeeds without a manual step; that is not yet done.

2.4.3 Path A: AWS Console

1. Open **EC2** → **Launch instance**. Name it `ssw590-group1-web`.
2. Choose the **Amazon Linux 2023** AMI (64-bit x86) and instance type **t3.micro**.

3. Under **Key pair**, create `ssw590-group1-key` (RSA, `.pem`) and save the downloaded file to `~/.ssh/`.
4. Under **Network settings**, create a security group that allows **SSH** from *My IP* and **HTTP** from *Anywhere*.
5. Expand **Advanced details**. In the **User data** box, paste the contents of `deploy/aws/ec2-user-data.sh`.
6. Click **Launch instance**, wait for *2/2 status checks passed*, and copy the **Public IPv4 DNS** from the instance page.

2.4.4 Path B: AWS CLI

The launch script creates the key pair and security group if they do not exist, looks up the latest Amazon Linux 2023 AMI from AWS's public SSM parameter, launches the instance with the bootstrap script, and prints the website address.

```
#!/bin/bash
# Launches one t3.micro (Free Tier eligible) Amazon Linux 2023 instance that
# runs ec2-user-data.sh at first boot. Requires the AWS CLI v2 configured with
# `aws configure` for an IAM user (never the root account).
set -euo pipefail

REGION="${AWS_REGION:-us-east-1}"
KEY_NAME="${KEY_NAME:-ssw590-group1-key}"
SG_NAME="${SG_NAME:-ssw590-group1-web}"
MY_IP="$(curl -s https://checkip.amazonaws.com)/32"
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"

# SSH key pair (private key stays on this machine, chmod 400)
if ! aws ec2 describe-key-pairs --region "$REGION" --key-names "$KEY_NAME" >/dev/null
↪  2>&1; then
  aws ec2 create-key-pair --region "$REGION" --key-name "$KEY_NAME" \
    --query KeyMaterial --output text > "$HOME/.ssh/$KEY_NAME.pem"
  chmod 400 "$HOME/.ssh/$KEY_NAME.pem"
fi

# Security group: HTTP from anywhere, SSH only from the current public IP
SG_ID=$(aws ec2 describe-security-groups --region "$REGION" \
  --filters Name=group-name,Values="$SG_NAME" \
  --query 'SecurityGroups[0].GroupId' --output text)
if [ "$SG_ID" = "None" ]; then
  SG_ID=$(aws ec2 create-security-group --region "$REGION" --group-name "$SG_NAME" \
    --description "SSW 590 group 1 web server" --query GroupId --output text)
  aws ec2 authorize-security-group-ingress --region "$REGION" --group-id "$SG_ID" \
    --protocol tcp --port 80 --cidr 0.0.0.0/0
  aws ec2 authorize-security-group-ingress --region "$REGION" --group-id "$SG_ID" \
    --protocol tcp --port 22 --cidr "$MY_IP"
fi
```

```
# Latest Amazon Linux 2023 AMI from the public SSM parameter
AMI_ID=$(aws ssm get-parameter --region "$REGION" \
  --name /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64 \
  --query Parameter.Value --output text)

INSTANCE_ID=$(aws ec2 run-instances --region "$REGION" \
  --image-id "$AMI_ID" --instance-type t3.micro \
  --key-name "$KEY_NAME" --security-group-ids "$SG_ID" \
  --user-data "file://$SCRIPT_DIR/ec2-user-data.sh" \
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ssw590-group1-web}]'
  ↪  \
  --query 'Instances[0].InstanceId' --output text)

aws ec2 wait instance-running --region "$REGION" --instance-ids "$INSTANCE_ID"
PUBLIC_DNS=$(aws ec2 describe-instances --region "$REGION" --instance-ids "$INSTANCE_ID"
↪  \
  --query 'Reservations[0].Instances[0].PublicDnsName' --output text)

echo "Instance: $INSTANCE_ID"
echo "Website (ready 2-3 minutes after boot): http://$PUBLIC_DNS"
```

It is run from the repository root:

```
git clone https://github.com/brahmi9811/ssw-590-group1.git
cd ssw-590-group1
./deploy/aws/launch-ec2.sh
```

2.4.5 Manual Commands on the Instance

For transparency, these are the same steps the bootstrap script automates, typed by hand after connecting over SSH. They are also the steps to follow if the user data is omitted.

```
chmod 400 ~/.ssh/ssw590-group1-key.pem
ssh -i ~/.ssh/ssw590-group1-key.pem ec2-user@<EC2-PUBLIC-DNS>

# on the instance
sudo dnf update -y
sudo dnf install -y docker git
sudo systemctl enable --now docker
sudo usermod -aG docker ec2-user && newgrp docker
docker --version

git clone https://github.com/brahmi9811/ssw-590-group1.git
cd ssw-590-group1/website/color-buttons-app
docker build -t color-buttons-app:2.0 .
docker run -d --name color-buttons-app --restart unless-stopped \
  -p 80:3000 color-buttons-app:2.0
docker ps
```

2.5 Verification

2.5.1 Pre-deployment Check on a Workstation

Before deploying, the exact image that the instance builds was built and run on a group member's workstation (Docker 29.4.2, Apple silicon), published on host port 8081. The output below is from that real run.

```
$ docker build -t color-buttons-app:2.0 .
#1 [internal] load build definition from Dockerfile
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/node:20-alpine
#2 DONE 5.6s
#3 [internal] load .dockerignore
#3 DONE 0.1s
#4 [internal] load build context
#4 DONE 0.0s
#5 [1/5] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d5
↪  0c58a04632e6c0fb8363f609372293
#5 DONE 0.0s
#6 [2/5] WORKDIR /app
#7 [3/5] COPY package*.json ./
#7 DONE 0.0s
#8 [4/5] RUN npm install --omit=dev
#8 DONE 1.9s
#9 [5/5] COPY . .
#9 DONE 0.0s
#10 naming to docker.io/library/color-buttons-app:2.0 done
#10 DONE 0.2s
$ docker run -d --name color-buttons-class -p 8081:3000 color-buttons-app:2.0
6b273f8872991a4c1561b6c696dfb43eeb4d4c59a8c245b0eb3a50fb4db75ec9
```

```
$ curl -s http://127.0.0.1:8081/health
{"status":"ok","uptimeSeconds":2}
$ curl -sI http://127.0.0.1:8081/js/colorButtons.js | head -4
HTTP/1.1 200 OK
X-Powered-By: Express
Accept-Ranges: bytes
Cache-Control: public, max-age=0
```

The page was then opened in a browser; clicking *Red* and then *Blue* changed the page background to red and then blue, served by the class-based code in Chapter 3.

2.5.2 Cloud Verification

After launch, the following commands confirm the deployment from any machine. Paste their output into `evidence/aws-checks.txt` and save a browser screenshot as `evidence/aws-website.png`.

```
curl -s http://<EC2-PUBLIC-DNS>/health        # {"status":"ok",...}
curl -sI http://<EC2-PUBLIC-DNS>/ | head -1   # HTTP/1.1 200 OK
ssh -i ~/.ssh/ssw590-group1-key.pem ec2-user@<EC2-PUBLIC-DNS> \
  'docker ps --format "{{.Names}} {{.Status}} {{.Ports}}"'
```

```
$ curl -s http://ec2-54-80-252-134.compute-1.amazonaws.com/health
{"status":"ok","uptimeSeconds":5}

$ curl -sI http://ec2-54-80-252-134.compute-1.amazonaws.com/ | head -1
HTTP/1.1 200 OK
```

A browser-only problem, and why port 443 is also open

`curl` reached the site over plain HTTP the whole time, but a browser pointed at the same `http://` address showed `ERR_CONNECTION_TIMED_OUT`. The cause was not the server: current Chrome and mobile browsers silently upgrade `http://` requests on `amazonaws.com` hosts to `https://` (an HSTS/HTTPS-first behavior that is hard-coded for many cloud domains and is not something a per-site browser setting can turn off), so the browser was actually trying port 443, which the security group did not yet open. The fix was to open port 443 (Table 2.1's security group) and put `nginx` in front of the container as a TLS-terminating reverse proxy on port 443, forwarding to the existing app on port 80. The connection now succeeds for any browser on any network. Figure 2.1 shows the deployed site in a browser, including the two-button click behavior working against the live cloud instance.

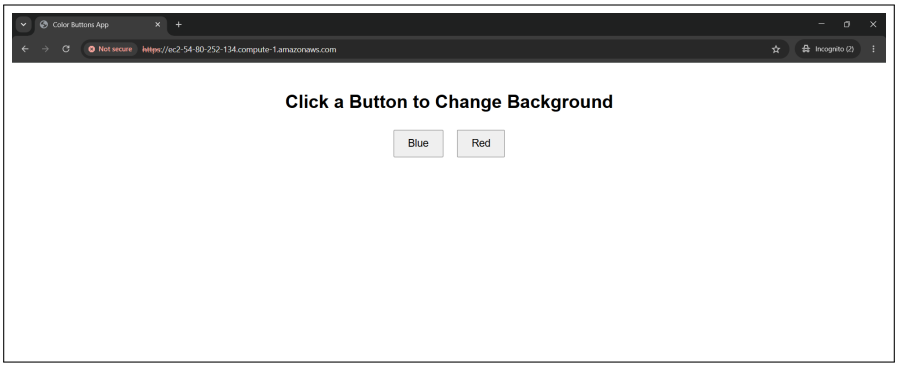


Figure 2.1: The deployed site at the public EC2 address, on load.

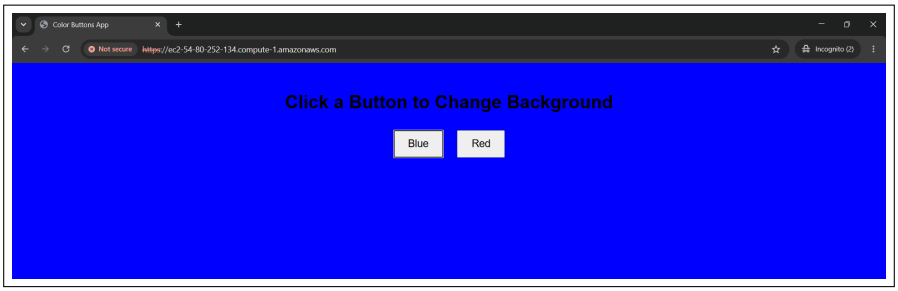


Figure 2.2: The same deployment after clicking *Blue*.

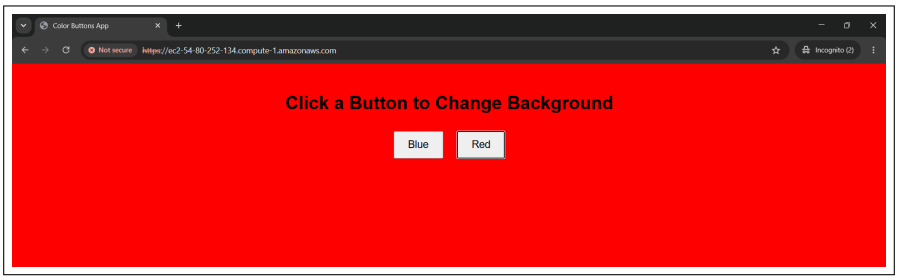


Figure 2.3: The same deployment after clicking *Red*. Together with Figures 2.1 and 2.2, this confirms the class-based code from Chapter 3 runs correctly on the real cloud deployment, not just locally.

Table 2.2: Deployment Checklist

Check	**Status**	**Evidence**
Image builds and runs locally	**PASS**	`evidence/local-build-run.txt`
`/health` and JS module served	**PASS**	`evidence/local-checks.txt`
Buttons work in a browser	**PASS**	Manual browser check (red, then blue)
EC2 instance launched	**PASS**	`i-00ee7edd4a5352f05`, public DNS `ec2-54-80-252-134.compute-1.amazonaws.com`
Public URL returns HTTP 200	**PASS**	`evidence/aws-checks.txt`
Screenshot of public site	**PASS**	`evidence/aws-website.png` (Figure 2.1)

2.6 Updating the Deployment

After a change is merged to `main`, a member connects over SSH and runs the redeploy script, which pulls the commit, rebuilds the image, and replaces the container:

```
#!/bin/bash
```

```
# Run on the EC2 instance to pull the latest commit and replace the container.
set -euo pipefail
cd /opt/ssw-590-group1
sudo git pull --ff-only
cd website/color-buttons-app
sudo docker build -t color-buttons-app:2.0 .
sudo docker rm -f color-buttons-app || true
sudo docker run -d --name color-buttons-app --restart unless-stopped -p 80:3000
↪  color-buttons-app:2.0
curl -s http://localhost/health
```

2.7 Security Notes

- SSH is open only to the launching member's public IP. HTTP (80) and HTTPS (443) are open to the internet; HTTPS was added, after deployment, for the reason explained in Section 2.4.5. The container itself listens on 3000, which is not exposed directly.
- No credentials are stored in the repository. The EC2 key pair's private key and the IAM access key remain on members' machines. The repository itself is private; the EC2 instance's own access to it is a read-only GitHub deploy key (Section 2.4.2) that was added specifically for this instance and grants no write access and no access to any other repository.
- The site is served over HTTPS using `nginx` with a **self-signed** certificate (Section 2.4.2 devopsNote), so browsers show a certificate warning that a user must click through. A production setup would replace this with a certificate from a real certificate authority (e.g. Let's Encrypt via `certbot`, or an Application Load Balancer with an ACM certificate), which requires owning a domain name the group did not acquire for this exercise.

2.8 Cost and Teardown

A `t3.micro` instance is Free Tier eligible for new accounts; outside the Free Tier it costs roughly one US cent per hour plus storage, and AWS also charges for public IPv4 addresses. When the site is no longer needed for grading, terminate it:

```
#!/bin/bash
# Terminates the group web server so it stops accruing charges.
set -euo pipefail
REGION="${AWS_REGION:-us-east-1}"
INSTANCE_ID=$(aws ec2 describe-instances --region "$REGION" \
  --filters Name=tag:Name,Values=ssw590-group1-web
  ↪  Name=instance-state-name,Values=running \
  --query 'Reservations[].Instances[].InstanceId' --output text)
[ -n "$INSTANCE_ID" ] && aws ec2 terminate-instances --region "$REGION" --instance-ids
↪  $INSTANCE_ID
```

Chapter 3

Class-Based Two-Button Website

3.1 Overview

The original website attached two anonymous arrow functions directly to the Blue and Red buttons. Each function repeated the same logic with a different color. This chapter replaces those inline handlers with three ES2015 classes in `public/js/colorButtons.js`, loaded by the page as a JavaScript module. The visible behavior is unchanged.

3.2 Original Design

```
<script>
  document.getElementById("blueBtn").addEventListener("click", () => {
    document.body.style.backgroundColor = "blue";
  });

  document.getElementById("redBtn").addEventListener("click", () => {
    document.body.style.backgroundColor = "red";
  });
</script>
```

3.3 Class-Based Design

- **`BackgroundChanger`** owns the element whose background changes (the page body) and is the only class that touches its style.
- **`ColorButton`** binds one button element to one color and delegates the change to a `BackgroundChanger`.
- **`ColorButtonApp`** holds the button configuration, creates one shared `BackgroundChanger` and one `ColorButton` per entry, and binds them. Adding a third color is a one-line change to `BUTTONS`.

3.3.1 UML Class Diagram

Figure 3.1 shows the updated design. A filled diamond marks composition: the application creates and owns its buttons and its background changer. The plain arrow is a directed association: each button holds a reference to the shared changer. An underlined attribute is static; `-` marks a field that is private by convention and `+` a public method. The same diagram is kept as Mermaid source in `docs/uml-class-diagram.md`, which GitHub renders directly.

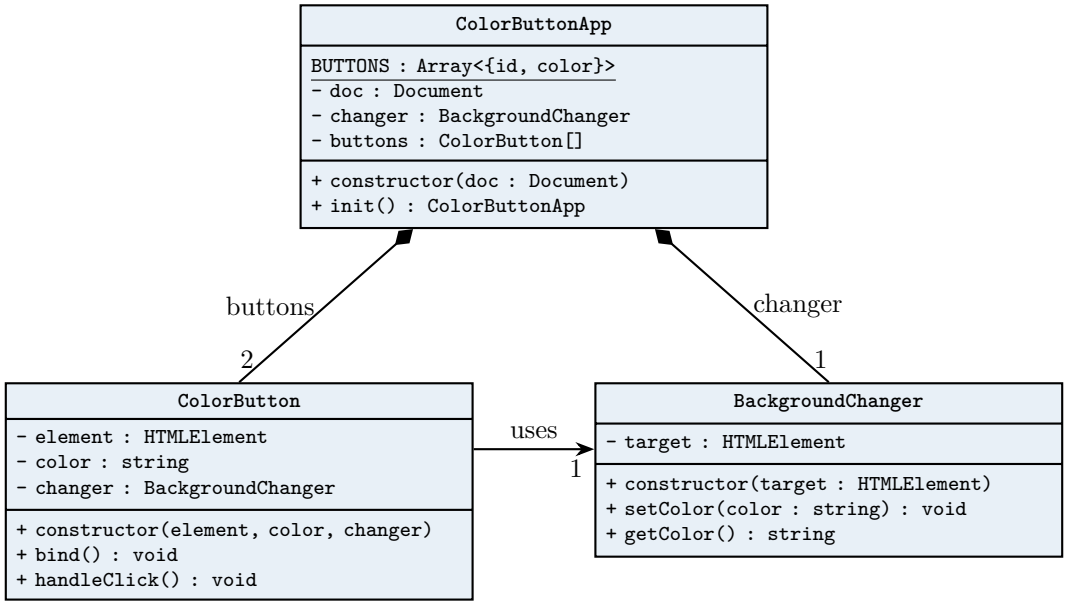


Figure 3.1: UML class diagram of the class-based two-button website.

3.4 Implementation

public/js/colorButtons.js — the three classes, exported as an ES module so the browser and the unit tests load the same file:

```
// Class-based design for the two-button website.
// BackgroundChanger owns the page background, ColorButton wires one button to
// one color, and ColorButtonApp creates and initializes both buttons.

export class BackgroundChanger {
  constructor(target) {
    this.target = target;
  }

  setColor(color) {
    this.target.style.backgroundColor = color;
  }
```

```
  getColor() {
    return this.target.style.backgroundColor;
  }
}

export class ColorButton {
  constructor(element, color, changer) {
    this.element = element;
    this.color = color;
    this.changer = changer;
  }

  bind() {
    this.element.addEventListener("click", () => this.handleClick());
  }

  handleClick() {
    this.changer.setColor(this.color);
  }
}

export class ColorButtonApp {
  static BUTTONS = [
    { id: "blueBtn", color: "blue" },
    { id: "redBtn", color: "red" },
  ];

  constructor(doc) {
    this.doc = doc;
    this.changer = new BackgroundChanger(doc.body);
    this.buttons = [];
  }

  init() {
    for (const { id, color } of ColorButtonApp.BUTTONS) {
      const button = new ColorButton(this.doc.getElementById(id), color, this.changer);
      button.bind();
      this.buttons.push(button);
    }
    return this;
  }
}
```

public/index.html — the page now loads the module and creates the application in two lines:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Color Buttons App</title>
  <style>
```

```
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
      transition: background-color 0.3s ease;
    }
    button {
      padding: 12px 24px;
      font-size: 18px;
      margin: 10px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h1>Click a Button to Change Background</h1>
  <button id="blueBtn">Blue</button>
  <button id="redBtn">Red</button>

  <script type="module">
    import { ColorButtonApp } from "./js/colorButtons.js";

    new ColorButtonApp(document).init();
  </script>
</body>
</html>
```

server.js — unchanged except for a `/health` endpoint used by the deployment checks in Chapter 2:

```
import express from "express";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT || 3000;

// Serve static files from public/
app.use(express.static(path.join(__dirname, "public")));

// Lightweight health check used by the deployment verification steps
app.get("/health", (req, res) => {
  res.json({ status: "ok", uptimeSeconds: Math.round(process.uptime()) });
});

app.listen(PORT, "0.0.0.0", () => {
  console.log(`Server running at http://0.0.0.0:${PORT}`);
});
```

Dockerfile — unchanged from the local Docker exercise. A new `.dockerignore` keeps `node_modules` and the tests out of the image.

```
FROM node:20-alpine

WORKDIR /app

# Install dependencies
COPY package*.json ./
RUN npm install --omit=dev

# Copy app code
COPY . .

# Expose port
EXPOSE 3000

# Run app
CMD ["npm", "start"]
```

3.5 Testing

Because the classes receive the document and elements through their constructors rather than reading globals, they can be tested in Node.js with small stand-in objects, without a browser. The tests use Node's built-in `node:test` runner, so no extra dependency is needed.

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { BackgroundChanger, ColorButton, ColorButtonApp } from
↪  "../public/js/colorButtons.js";

// Minimal stand-ins for the browser DOM objects the classes use.
class FakeElement {
  constructor() {
    this.style = {};
    this.listeners = {};
  }
  addEventListener(type, fn) {
    this.listeners[type] = fn;
  }
  click() {
    this.listeners.click?.();
  }
}

function fakeDocument() {
  const elements = { blueBtn: new FakeElement(), redBtn: new FakeElement() };
  return { body: new FakeElement(), getElementById: (id) => elements[id] };
}
```

```
test("BackgroundChanger sets and reports the color", () => {
  const changer = new BackgroundChanger(new FakeElement());
  changer.setColor("green");
  assert.equal(changer.getColor(), "green");
});

test("ColorButton applies its color when clicked", () => {
  const element = new FakeElement();
  const changer = new BackgroundChanger(new FakeElement());
  new ColorButton(element, "blue", changer).bind();
  element.click();
  assert.equal(changer.getColor(), "blue");
});

test("ColorButtonApp wires the Blue and Red buttons", () => {
  const doc = fakeDocument();
  const app = new ColorButtonApp(doc).init();
  assert.equal(app.buttons.length, 2);
  doc.getElementById("blueBtn").click();
  assert.equal(doc.body.style.backgroundColor, "blue");
  doc.getElementById("redBtn").click();
  assert.equal(doc.body.style.backgroundColor, "red");
});
```

Result of `npm test`:

```
$ npm test
> color-buttons-app@2.0.0 test
> node --test test/*.test.js
✓ BackgroundChanger sets and reports the color (0.602292ms)
✓ ColorButton applies its color when clicked (0.068375ms)
✓ ColorButtonApp wires the Blue and Red buttons (0.081584ms)
ℹ tests 3
ℹ suites 0
ℹ pass 3
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 71.673458
```

The module was also loaded in a real browser from the running container, where clicking *Red* then *Blue* set the body background to `red` and then `blue`.

Chapter 4

LaTeX Docker

4.1 Overview

Overleaf compiles every project with TeX Live, run inside containers on its servers. This chapter reproduces that setup locally: a Docker image built on the official `texlive/texlive` image compiles a simple LaTeX document from a folder on the host. The same container builds this group document, so every member gets an identical toolchain without installing MacTeX or MiKTeX, and the build can later run unchanged in a CI pipeline.

4.2 How It Works

1. The image contains a full TeX Live installation plus `latexmk`, which reruns `pdflatex`, BibTeX, and index tools until cross-references settle.
2. At run time the current host folder is bind-mounted at `/workdir`, so the container reads the `.tex` source and writes the PDF back to the host.
3. The container is started with `-rm`, so it is deleted after each compile. Only the image persists.

4.3 Project Files

```
latex-docker/
  Dockerfile    image definition
  hello.tex     simple test document
  build.sh      builds the image and compiles a .tex file
  .gitignore    keeps auxiliary files out of Git
```

Dockerfile — `ENTRYPOINT` fixes the compiler and flags, while `CMD` supplies a default file name that can be overridden on the `docker run` line. `-shell-escape` is required by `minted`; it lets LaTeX call the Pygments highlighter, so it should only be used on trusted documents.

```
# Compiles LaTeX documents with TeX Live, the same distribution Overleaf uses.
FROM texlive/texlive:latest

# Documents are mounted here from the host at run time
WORKDIR /workdir

# latexmk reruns pdflatex/bibtex/makeindex until references settle.
# -shell-escape is required by the minted package used in the group report.
ENTRYPOINT ["latexmk", "-pdf", "-shell-escape", "-interaction=nonstopmode",
↪  "-halt-on-error"]
CMD ["hello.tex"]
```

hello.tex — a small document that exercises text, math, and a `minted` code block:

```
\documentclass{article}
\usepackage{amsmath}
\usepackage{minted}

\title{Hello from a TeX Live Container}
\author{SSW 590 Group 1}
\date{\today}

\begin{document}
\maketitle

\section{Introduction}
This document was compiled inside a Docker container built from the
\texttt{texlive/texlive} image, the same \TeX{} Live distribution that
Overleaf uses to compile projects.

\section{Some Math}
\begin{equation}
  e^{i\pi} + 1 = 0
\end{equation}

\section{Some Code}
\begin{minted}{bash}
docker run --rm -v "$PWD":/workdir latex-builder hello.tex
\end{minted}

\end{document}
```

build.sh — one command to build and compile:

```
#!/bin/bash
# Build the LaTeX image once, then compile a .tex file from the current folder.
# Usage: ./build.sh [file.tex]   (defaults to hello.tex)
set -euo pipefail
cd "$(dirname "$0")"
```

```
docker build -t latex-builder .
docker run --rm -v "$PWD":/workdir latex-builder "${1:-hello.tex}"
```

4.4 Commands Run

4.4.1 Build the Image

The base image is large (about 2.7 GB compressed) because it contains the complete TeX Live distribution, the same scheme Overleaf offers. It is pulled once and then cached.

```
cd latex-docker
docker pull texlive/texlive:latest
docker build -t latex-builder .
```

```
$ docker build -t latex-builder .
#1 [internal] load build definition from Dockerfile
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/texlive/texlive:latest
#2 DONE 0.0s
#3 [internal] load .dockerignore
#3 DONE 0.0s
#4 [1/2] FROM docker.io/texlive/texlive:latest@sha256:7334b00bf8e7a0996f7ddd65482363aaf7
↪  711d372e569f3ea78509619e3083ff
#4 DONE 0.0s
#5 [2/2] WORKDIR /workdir
#5 DONE 0.0s
#6 naming to docker.io/library/latex-builder:latest done
#6 DONE 0.1s
```

4.4.2 Compile the Document

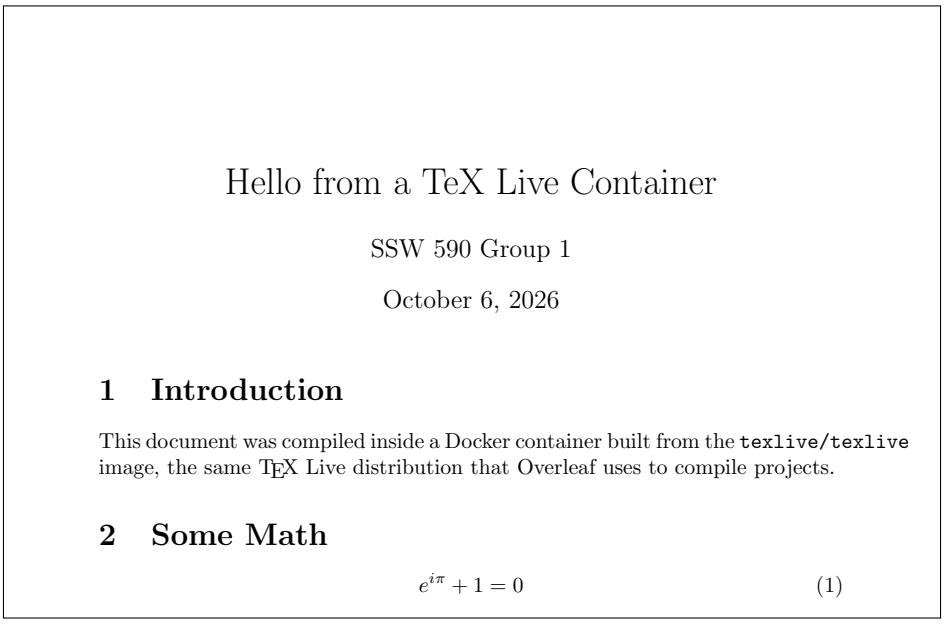
```
docker run --rm -v "$PWD":/workdir latex-builder hello.tex
```

The output below is abridged to `latexmk`'s run summary; the TeX Live 2026 `pdflatex` ran three times before reporting that the PDF was up to date.

```
$ docker run --rm -v "$PWD":/workdir latex-builder hello.tex
Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
Run number 1 of rule 'pdflatex'
Running 'pdflatex  -shell-escape -interaction=nonstopmode -halt-on-error -recorder
↪  "hello.tex"'
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded
↪  format=pdflatex)
Output written on hello.pdf (1 page, 79481 bytes).
```

```
Run number 2 of rule 'pdflatex'
Running 'pdflatex  -shell-escape -interaction=nonstopmode -halt-on-error -recorder
↪  "hello.tex"'
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded
↪  format=pdflatex)
Output written on hello.pdf (1 page, 81626 bytes).
Run number 3 of rule 'pdflatex'
Running 'pdflatex  -shell-escape -interaction=nonstopmode -halt-on-error -recorder
↪  "hello.tex"'
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded
↪  format=pdflatex)
Output written on hello.pdf (1 page, 81626 bytes).
Latexmk: All targets (hello.pdf) are up-to-date
$ ls -l hello.pdf
-rw-r--r-- 81626 hello.pdf
```

Figure 4.1 shows the resulting PDF.



Hello from a TeX Live Container

SSW 590 Group 1

October 6, 2026

1 Introduction

This document was compiled inside a Docker container built from the `texlive/texlive` image, the same TEX Live distribution that Overleaf uses to compile projects.

2 Some Math

$$e^{i\pi} + 1 = 0 \tag{1}$$

Figure 4.1: Top of `hello.pdf` compiled by the `latex-builder` container.

4.4.3 Compile This Group Document

The same image builds this report. From the repository root, the whole repository is mounted so the report can include source files from `website/`, `deploy/`, and `evidence/`:

```
docker run --rm -v "$PWD":/workdir -w /workdir/report \
  latex-builder devopsAssignment.tex
```

4.5 Comparison with Overleaf

Table 4.1: Overleaf Compared with the LaTeX Container

Aspect	**Overleaf**	**LaTeX Docker**
Engine	TeX Live in server-side containers	TeX Live in a local container
Collaboration	Real-time editing and comments	Git branches and pull requests
Version control	Built-in history; Git sync on paid plans	Native Git in the group repo
Compile limits	Timeout on free plan	Limited only by the machine
Offline use	No	Yes, once the image is pulled
Automation	Manual recompile	Scriptable; reusable in CI (Chapter 5)

4.6 Troubleshooting

- **Apple silicon.** `texlive/texlive` publishes `arm64` images, so it runs natively on Apple silicon Macs without emulation.
- **File not found.** The `.tex` file must be inside the mounted folder; paths are resolved relative to `/workdir`.
- **Glossaries.** `latexmk` does not run `makeglossaries` by default; `report/latexmkrc` adds that rule for this document.

Chapter 5

Project Proposal

5.1 Title

TwinTrack: A Side-by-Side DevSecOps Comparison of GitHub Actions and Jenkins Delivering a Containerized Task Tracker to AWS

5.2 Project Description

Group 1 will build one small but complete web application, *TwinTrack*, a team task tracker, and deliver it to AWS through **two parallel DevSecOps pipelines** built with comparable tools: **GitHub Actions** (hosted, YAML-defined) and **Jenkins** (self-hosted, Groovy `Jenkinsfile`). Both pipelines start from the same commit in the same GitHub repository, run the same tests and security scans, build the same Docker image, and deploy it to identical AWS environments ("blue" for GitHub Actions, "green" for Jenkins). Because the application, tests, and target are held constant, differences in speed, reliability, security coverage, cost, and operator effort can be attributed to the pipeline tool itself.

An operations dashboard will collect metrics from both pipelines and both running environments, and the group will finish with a modified strengths, weaknesses, opportunities, and threats (SWOT) chart for each tool and a recommendation for which tool suits which kind of team.

5.2.1 The Application

TwinTrack is deliberately simple so the effort goes into DevOps rather than features:

- A REST API (Node.js and Express, the stack from the two-button site) to create, list, update, and close tasks, with a `/health` endpoint and a Prometheus `/metrics` endpoint.
- A small browser front end written with class-based JavaScript, following the design in Chapter 3.

- A PostgreSQL database for tasks and users.

5.2.2 Sample Tasks

Table 5.1: Sample Project Tasks

ID	Task	Area
T1	Set up the GitHub repository, branch protection on `main`, pull-request template, and GitHub Projects Kanban board	Source control
T2	Build the TwinTrack API and front end with Docker Compose for local development (app + PostgreSQL)	Development
T3	Write unit, API, and end-to-end tests; enforce a coverage threshold	Testing
T4	Provision two identical AWS environments (blue and green) with Terraform	Infrastructure as code
T5	Write the GitHub Actions workflow: lint, test, scan, build, push to Amazon ECR, deploy to blue	CI/CD tool 1
T6	Run Jenkins in Docker on EC2 and write an equivalent `Jenkinsfile` that deploys to green	CI/CD tool 2
T7	Add security gates to both pipelines: secret scanning, dependency audit, container image scanning, and a DAST baseline scan	Security
T8	Build the operations dashboard in Grafana (pipeline and runtime metrics) and compare with Amazon CloudWatch	Monitoring
T9	Run the planted-fault experiment (Section 5.9) against both pipelines	Experiment
T10	Write SWOT charts, comparison tables, and the final report; build the report with the LaTeX container from Chapter 4	Documentation

5.3 DevSecOps Tools

At this stage the list is intentionally broad; tools may be swapped as the group learns more. Pairs marked “vs.” are candidates for head-to-head comparison.

Table 5.2: Planned DevSecOps Tool Chain

Category	Tool(s)	Role
Source control	Git, GitHub (branch protection, pull requests, CODEOWNERS)	Single source of truth for both pipelines

Category	**Tool(s)**	**Role**
Planning	GitHub Projects Kanban board	Track tasks T1–T10
CI/CD (**primary comparison**)	**GitHub Actions vs. Jenkins**	Build, test, scan, deploy
Containers	Docker, Docker Compose	Package and run app and database
Image registry	Amazon ECR	Store versioned images
Infrastructure as code	Terraform	Create identical blue and green environments
Hosting	AWS EC2 (Docker on Amazon Linux 2023); possible stretch to ECS Fargate	Run the application
Database	PostgreSQL (container locally, Amazon RDS in AWS)	Persist tasks
Unit and API testing	Node `node:test` or Jest, Supertest	Fast feedback on every commit
End-to-end testing	Playwright	Test the UI in a real browser
Load testing	k6	Measure latency under load
Secret scanning	Gitleaks	Block committed secrets
Dependency scanning	`npm audit`, GitHub Dependabot	Find vulnerable packages
Image scanning	Trivy	Find OS and library CVEs in images
Static analysis	ESLint, GitHub CodeQL	Code quality and security bugs
Dynamic analysis	OWASP ZAP baseline scan	Test the running site
Secrets management	GitHub Encrypted Secrets vs. Jenkins Credentials; AWS IAM roles	Keep keys out of code
Monitoring (**secondary comparison**)	Prometheus + Grafana vs. Amazon CloudWatch	Runtime and pipeline metrics

Category	Tool(s)	Role
Documentation	LaTeX in Docker (Chapter 4)	Reproducible report builds

5.4 Pipeline Design

Both pipelines implement the same stages so they can be compared stage by stage:

```
commit/PR -> checkout -> secret scan (Gitleaks) -> install -> lint (ESLint)
  -> unit + API tests -> dependency audit -> docker build -> image scan (Trivy)
  -> push to ECR -> deploy to blue|green -> smoke test (/health)
  -> DAST (ZAP baseline) -> publish metrics
```

A pipeline fails if any security gate reports a high or critical finding. Pipeline timing, result, and finding counts are exported to the dashboard.

5.5 Environment

- **Cloud:** AWS, region `us-east-1`, using the account, security model, and scripts established in Chapter 2.
- **Blue environment:** EC2 instance deployed by GitHub Actions through an OpenID Connect (OIDC) role, so no long-lived AWS keys are stored in GitHub.
- **Green environment:** EC2 instance deployed by Jenkins, which runs in its own Docker container on a separate EC2 instance with an IAM instance role.
- **Local:** Docker Desktop and Docker Compose on members' laptops.
- **Budget:** Free Tier instances where possible, an AWS Budget alert, and Terraform `destroy` after each work session.

5.6 Operations Dashboard

A Grafana dashboard will place both pipelines and both environments side by side. Planned panels:

- **DevOps Research and Assessment (DORA) metrics per tool:** deployment frequency, lead time for changes, change failure rate, and time to restore service.
- **Pipeline health:** median and 95th-percentile duration per stage, queue time, and success rate.

- **Security:** open findings by scanner and severity, and time from introducing a planted fault to the pipeline blocking it.
- **Runtime:** request rate, error rate, and latency from the app's `/metrics` endpoint; CPU and memory of each instance.
- **Cost:** estimated cost per deployment and monthly cost per environment (from AWS Cost Explorer).

The same signals will be reproduced in a CloudWatch dashboard to compare the managed AWS option with the self-hosted Prometheus and Grafana option.

5.7 Comparison Criteria

Table 5.3: How the Two CI/CD Tools Will Be Compared

Dimension	Measure
Security	Planted faults caught; secret-handling model; attack surface of the CI server; supply-chain risk of plugins or marketplace actions
Development	Lines of pipeline code; time to write and debug the pipeline; local testability
Hosting	Who runs the CI server; setup time; upgrade effort
Monitoring	Metrics available out of the box; effort to export to Grafana
Testing	Ease of parallel test jobs and test-report publishing
Operations	Pipeline duration; failure and recovery time; maintenance hours per week
Cost	AWS and CI minutes per deployment; cost at the group's scale and projected at a larger team's scale

5.8 Preliminary Modified SWOT Charts

The "modified" SWOT used here keeps the four classic quadrants but scores each one from the perspective of a small DevSecOps team and adds a final *Evidence to collect* row, so every claim is tested during the project rather than assumed. These are initial hypotheses and will be revised with measured results in the final report. The four charts are Tables 5.4–5.7.

5.9 What Makes This Project Unique

1. **Controlled "twin" experiment.** The same commit, tests, image, and AWS target run through both tools at the same time, so the comparison is measured rather than anecdotal.

Table 5.4: Preliminary Modified SWOT: GitHub Actions

Strengths No server to run; tight GitHub integration; large marketplace; OIDC to AWS without stored keys.	**Weaknesses** Vendor lock-in to GitHub; limited free minutes for private repos; harder to debug locally.
Opportunities Reusable workflows across courses; CodeQL and Dependabot built in; matrix builds.	**Threats** Compromised third-party actions (supply chain); outages outside the team's control.
Evidence to collect: Pipeline duration, minutes used, planted-fault detection time, lines of YAML.	

Table 5.5: Preliminary Modified SWOT: Jenkins

Strengths Full control and no usage limits; mature plugin ecosystem; runs anywhere, including air-gapped networks.	**Weaknesses** Team must host, patch, and secure the server; plugin version conflicts; Groovy learning curve.
Opportunities Deep customization; shared libraries; on-premises compliance scenarios.	**Threats** Unpatched server or plugins as an attack surface; credential leakage from misconfigured jobs.
Evidence to collect: Setup and maintenance hours, server cost, CVEs in installed plugins, pipeline duration.	

Table 5.6: Preliminary Modified SWOT: Prometheus and Grafana

Strengths Open source; powerful queries; one dashboard for both pipelines and both environments.	**Weaknesses** Another service to host and secure; storage retention must be managed.
Opportunities Custom DORA panels; alerting to Slack or e-mail.	**Threats** Dashboard exposed publicly without authentication; metric gaps if exporters fail.
Evidence to collect: Setup time, resource use, number of signals shown without custom code.	

Table 5.7: Preliminary Modified SWOT: Amazon CloudWatch

Strengths Managed; EC2 metrics with no setup; integrated alarms.	**Weaknesses** Per-metric and per-dashboard cost; weak view of CI pipeline data; AWS-only.
Opportunities Logs Insights for incident analysis; native alarms for auto-recovery.	**Threats** Costs grow with custom metrics; lock-in to AWS.
Evidence to collect: Monthly cost, setup time, ability to show pipeline metrics.	

2. **Planted-fault security drill.** On a test branch, the group will deliberately introduce known problems one at a time: a fake (non-functional) AWS key in source, a dependency with a published CVE, a base image with known vulnerabilities, and a missing security header. The dashboard records which pipeline catches each fault, at which stage, and how long it takes. This turns "which tool is more secure?" into a scored result.

3. **Cost per deployment.** Each successful deployment is tagged with its CI time and AWS resources, giving a cost-per-deploy figure for each tool.

4. **Docs as code.** The final report is built from the repository by the LaTeX container, so the report itself can be a pipeline artifact.

5.10 Mapping to the Project Rubric

Table 5.8: Proposal Coverage of the Project Expectations

Expectation	**How TwinTrack addresses it**
Selected tools and environment	Table 5.2; AWS plus local Docker
Sample working application	TwinTrack API, front end, and PostgreSQL database
Specific DevOps tools to build it	Git/GitHub, Docker, Terraform, Trivy, Gitleaks, ZAP, Playwright, k6
Two comparable tools	GitHub Actions vs. Jenkins (primary); Prometheus/Grafana vs. CloudWatch (secondary)
Comparison across security, development, hosting, monitoring, testing, operations	Table 5.3
Environment (AWS, Digital Ocean, or homebrew)	AWS EC2, ECR, RDS; Docker locally

Expectation	How TwinTrack addresses it
Operational dashboard	Grafana dashboard with DORA, pipeline, security, runtime, and cost panels
Modified SWOT chart for each tool	Preliminary charts above; final charts with measured evidence
Something unique	Twin experiment, planted-fault drill, cost per deploy, docs as code

5.11 Team Roles and Timeline

Member	Primary responsibility
Brahmi Dalvi	Application, tests, GitHub Actions pipeline, Grafana dashboard
Sakshi Deshpande	Terraform/AWS environments, Jenkins pipeline, security gates, report build
Both	Planted-fault experiment, SWOT analysis, final report and presentation

Weeks	Milestone
1–2	Repository, Kanban board, app skeleton, Docker Compose (T1–T2)
3–4	Tests and Terraform environments (T3–T4)
5–6	GitHub Actions and Jenkins pipelines with security gates (T5–T7)
7	Dashboard and planted-fault experiment (T8–T9)
8	Final SWOT charts, report, and presentation (T10)

Bibliography

[1] G. Kim, K. Behr, and G. Spafford, *The Phoenix Project: A Novel about IT, DevOps, and Helping Your Business Win*. IT Revolution Press, 2018.

Appendix A

Document Authoring Guide

A.1 How to Use This Appendix

Use this appendix to maintain the template's file conventions, build process, and review rules. Update it when the document structure or toolchain changes.

A.2 Template Provenance

This template was developed by **Darian Muresan, Ph.D.**, in conjunction with **OpenAI Codex** and **Anthropic Claude Code**. Keep this attribution distinct from the assignment-specific instructor metadata.

A.3 File Naming and Responsibilities

Every chapter file shall use the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. Each such file shall contain one complete chapter, beginning with `\chapter{...}`, and all of its sections and subsections. Do not split one chapter into section-level `.tex` files. For example, all of Chapter 2 is contained in `devopsExampleAssignment.tex`. Do not add requirements or architecture-design chapters; this template records a DevOps learning activity and its operational evidence.

A.4 Synonyms and Controlled Terms

Use one preferred term for each important concept. If an assignment uses equivalent terms, record them here before release. For example, state whether "continuous delivery" refers to continuous delivery or continuous deployment (CD) generally or specifically excludes automatic production deployment. Add important terms to `devopsGlossary.tex` and index them where used.

A.5 Building the PDF

The template requires a LaTeX distribution with `pdflatex`, `bibtex`, `makeglossaries`, and `makeindex`. Code listings use the `minted` package, so LaTeX must also run with the `-shell-escape` flag and Python Pygments must be installed (`pip install Pygments`).

A.5.1 Group Repository Build (Docker)

This group document is built with the TeX Live container described in Chapter 4, so no local LaTeX installation is required. From the repository root, run:

```
docker build -t latex-builder latex-docker
docker run --rm -v "$PWD":/workdir -w /workdir/report \
  latex-builder devopsAssignment.tex
```

`latexmk` reruns `pdflatex`, `bibtex`, `makeglossaries` (through `report/latexmkrc`), and `makeindex` until all references settle. The PDF is written to `report/devopsAssignment.pdf`. Review the log and visually inspect affected PDF pages. Do not edit generated auxiliary files or the PDF directly.

A.5.2 Local Toolchain Build

A member who prefers a local toolchain can install the tools using the platform instructions below and run the same `latexmk` command from the `report` folder:

```
latexmk -pdf -shell-escape devopsAssignment.tex
```

A.6 Install LaTeX Tools on macOS

A.6.1 Recommended Installation

Install the current full MacTeX distribution. It includes TeX Live, `pdflatex`, `bibtex`, `latexmk`, `makeglossaries`, `makeindex`, and useful applications such as TeXShop and TeX Live Utility. The full distribution is the least troublesome choice for this template because it includes the packages used by the glossary, table, listing, color, and hyperlink features.

Either download the signed installer from the MacTeX web site and complete the macOS installation dialog, or, if Homebrew is already installed, run:

```
brew install --cask mactex
```

Close and reopen Terminal after installation. If the TeX commands are not found, add the MacTeX binary directory to the current shell session:

```
export PATH="/Library/TeX/texbin:$PATH"
```

Do not place this command in a shell profile until it has been verified on the local computer.

A.6.2 Verification and Compilation

From the template directory, verify the tools:

```
command -v pdflatex bibtex latexmk makeglossaries makeindex
```

Each command should print a path. Then create the PDF:

```
latexmk -pdf -shell-escape devopsAssignment.tex
```

To remove generated intermediate files while keeping the PDF, run:

```
latexmk -c devopsAssignment.tex
```

A.7 Install LaTeX Tools on Windows

A.7.1 Recommended Installation

Install the current Basic MiKTeX distribution using the official MiKTeX Windows installation guide. During installation, choose a per-user installation when appropriate and set missing-package installation to *Ask me first* or *Yes*. The latter is convenient for a student machine with Internet access; the former makes each package installation explicit.

Open MiKTeX Console after installation, apply available updates, and install the `latexmk` package if the verification command below cannot find it. On an organization-managed computer, obtain approval before installing software or changing execution policy.

A.7.2 Verification and Compilation

Open a new PowerShell window in the template directory and verify:

```
Get-Command pdflatex,bibtex,latexmk,makeglossaries,makeindex
```

Each command should display a command record. Build the PDF with:

```
latexmk -pdf -shell-escape devopsAssignment.tex
```

If PowerShell blocks the script under a managed execution policy, ask the system administrator for the approved procedure. Do not bypass institutional security policy. To remove intermediate build files, run:

```
latexmk -c devopsAssignment.tex
```

A.8 Common Build Problems

Symptom	Resolution
Command not found	Close and reopen the terminal. On macOS, verify the TeX binary path; on Windows, verify MiKTeX completed installation and is on the user PATH.
Missing LaTeX package	Use MiKTeX Console or its approved on-the-fly package installation prompt on Windows. On macOS, install the full MacTeX distribution or use TeX Live Utility to add the missing package.
References or glossary incomplete	Run `latexmk` (or the Docker build) instead of a single `pdflatex` command; it runs BibTeX, the glossary tool, and the index tool in the required order.
minted error such as "You must invoke LaTeX with the -shell-escape flag"	Run `pdflatex` with `-shell-escape`. If the error names Pygments or `latexminted`, install Pygments with `pip install Pygments`.
Script is blocked	Follow the institution's approved PowerShell policy or run the documented build commands with an approved administrator.

A.9 Authoring and Review Rules

Replace bracketed placeholders with assignment facts. Keep operational examples safe, bounded, and free of credentials. Link activities to evidence, retain history, and state unknowns or instructor decisions explicitly. Rebuild after structural, cross-reference, table, glossary, or index changes.

Appendix B

AI Agent Usage Guide

B.1 How to Use This Appendix

An AI agent that creates, reviews, or maintains an assignment shall read this file first. It shall identify the task in the routing table, read the minimum authoritative sources, and expand context only for a discovered dependency or inconsistency. This guide is maintained in LaTeX so it is visible, versioned, and reviewable with the assignment.

B.2 Governing Principles

The authored LaTeX files, versioned automation, and approved assignment materials are authoritative. The PDF is a review artifact and shall not be edited directly. AI may draft, explain, analyze, check consistency, and assist with builds. It shall not invent completion, evidence, access, costs, grades, approvals, command output, security posture, or operational results.

B.3 Chapter File Convention

Every assignment chapter shall be stored in exactly one file named with the `devops` prefix, a PascalCase chapter name, and the `.tex` suffix. For example, Chapter 2 is fully contained in `devopsExampleAssignment.tex`. A chapter file shall begin with its `\chapter{...}` command and contain all of that chapter's sections and subsections. Do not split one chapter across files, create section-only `.tex` files, or use `\input` or `\include` to assemble a chapter from smaller fragments.

Supporting non-chapter files may use a different name only when they are not chapter content: `prologue.tex`, `abstract.tex`, `devopsGlossary.tex`, and `devopsReferences.bib` are examples. Appendix chapters follow the same chapter-file convention.

B.4 AI Quick Start

1. Read this guide and identify the requested outcome.
2. Select the closest task below and load only its listed sources.
3. Inspect supplied facts and report consequential gaps or assumptions.
4. Make narrowly scoped changes; preserve stable IDs, glossary terms, and history.
5. Never request, print, store, or commit credentials, tokens, personal data, or production data.
6. Build the complete PDF, inspect the log, visually inspect affected pages, and record material AI use in Chapter C.

B.5 Task Routing

Table B.1: AI Task-Routing Table

Task	**Minimum authoritative sources**
Initialize an assignment	`devopsAssignment.tex`, `prologue.tex`, `devopsIntroduction.tex`, `devopsExampleAssignment.tex`, and the instructor's assignment facts.
Develop a workflow	`devopsExampleAssignment.tex`, source repository materials, and authorized environment constraints.
Define evidence	`devopsExampleAssignment.tex` and controlled execution outputs.
Review a retrospective	`devopsExampleAssignment.tex` and the submitted workflow or runbook.
Edit AI guidance	`devopsAIAgentGuide.tex`, `devopsAIPromptLog.tex`, `devopsDocumentGuide.tex`, and `devopsAssignment.tex`.
Compile or troubleshoot	`devopsAssignment.tex`, `devopsDocumentGuide.tex`, the affected chapter, and the build log.

B.6 DevOps-Specific Rules

1. Prefer versioned, repeatable automation over undocumented manual steps.
2. Treat deployment output, monitoring data, and screenshots as evidence only when they reflect an actual execution.
3. Design failure exercises to be isolated, reversible, and authorized.
4. Apply least privilege and never place secrets in LaTeX, source control, prompts, screenshots, or logs.

5. State expected cloud cost, cleanup action, and resource ownership for each remote-environment activity.

6. Identify whether a recommendation is a proposal, an instructor directive, or demonstrated evidence.

B.7 Example AI Requests

B.7.1 Create an Assignment Brief

```
Read devopsAIAgentGuide.tex and use the Initialize an Assignment context.
From the supplied SSW 590 objective and constraints, fill only supported
placeholders in the introduction and activity brief. Propose clear, observable
completion criteria, a safe cleanup action, and a rubric. Do not invent cloud
access, due dates, policy, costs, or approved tools. Update history, compile,
and report instructor decisions still needed.
```

B.7.2 Review a Student Workflow

```
Read devopsAIAgentGuide.tex and the workflow, evidence, and retrospective
chapters. Review the supplied repository and sanitized evidence for
reproducibility, least privilege, observability, rollback, and cleanup.
Separate verified facts from gaps and recommendations. Do not claim that an
unexecuted command passed, and do not request credentials or operate resources.
```

B.8 Human Decisions and Information Boundaries

The instructor retains authority over assignment scope, grading, due dates, approved tooling, access, budgets, security posture, exceptions, and policy. An AI agent shall surface alternatives and evidence, not represent a decision as approved. Record material prompts, assumptions, corrections, and unresolved issues in Chapter C.

B.9 Completion Checklist

- Scope, deliverables, environment, constraints, and cleanup are clear.
- Activity identifiers and evidence references are valid and unique.
- Examples contain no secrets, personal data, or unsafe production actions.
- New controlled terms are added to the glossary and indexed.
- The newest approved change is recorded in document history.
- The complete build succeeds with no unresolved references or missing files.

- A human reviews all decisions requiring instructional or operational authority.

Appendix C

AI Prompt Log

C.1 How to Use This Appendix

Use this log when AI assistance materially affects assignment wording, automation, evidence interpretation, troubleshooting, or feedback. Record enough context for an instructor or reviewer to understand the input, output, assumptions, and human corrections. Do not record credentials or personal data.

C.2 Record: Group Document, AWS Deployment, LaTeX Docker, and Proposal

Date: October 6, 2026; Tool and model: Claude Code (Claude Opus 5.5); Task: create the Group 1 document in the group repository and complete the "Digital Ocean Docker + Project Proposal" assignment using AWS instead of Digital Ocean; Inputs: the assignment text, the group's decision to use AWS, and Sakshi Deshpande's individual document and two-button website from the Local Docker exercise; Output used: the class-based refactor of the website with unit tests, the AWS deployment scripts under `deploy/aws`, the `latex-docker` container, the four new chapters, the UML diagram, and the updated document history. The assistant ran the unit tests, built and ran the website image locally, checked the buttons in a browser, and built the LaTeX image and compiled `hello.tex`; that output is stored in `evidence/`. In this session the assistant did *not* create AWS resources: it had no AWS account access, and launching paid infrastructure was deferred to a session where a group member could configure the AWS CLI (see the next record). Human review and corrections: Brahmi Dalvi and Sakshi Deshpande reviewed the drafted chapters together; the AWS-for-Digital-Ocean substitution (disclosed in Chapter 2) and the TwinTrack project proposal's scope (Chapter 5) were discussed and both confirmed as-is, with the group's intent to pace the proposal's ambitious scope across the semester rather than trim it now. Unresolved assumptions at the time of this record – the public EC2 address, the AWS evidence files, the due date, and the team-role split – were all resolved in the next record.

C.3 Record: AWS Deployment Execution and Report Fixes

Date: October 6–7, 2026; Tool and model: Claude Code (Claude Sonnet 5); Task: execute the AWS EC2 deployment that the previous record only scripted, using Brahmi Dalvi's AWS account. Inputs: the group repository at the state left by the previous record, live AWS CLI access configured by Brahmi Dalvi on her own machine, and her real-time command output and browser screenshots throughout. Output used: a running `t3.micro` EC2 instance, real command output saved to `evidence/aws-checks.txt`, and three real browser screenshots (`evidence/aws-website*.png`). The assistant *did* create real AWS resources this session (an EC2 instance, a key pair, and security group rules) and real changes on the instance itself (an nginx TLS reverse proxy), each time after the group member's live authorization to proceed; the assistant never handled raw AWS access keys or the GitHub deploy key's private half. One class of action – installing software and writing configuration over SSH on the EC2 instance – was blocked by the assistant's own sandbox and was run by Brahmi Dalvi herself from commands the assistant supplied; the assistant verified the results afterward with read-only AWS CLI calls and `curl`. Along the way the assistant diagnosed and fixed three real problems documented in Chapter 2: the bootstrap script's `git clone` failing against the group's private repository, a browser-only HTTPS/HSTS timeout that `curl` did not reproduce, and an `nginx` default server block silently claiming port 80. It also fixed two defects in the document build itself: a `tcolorbox`/`pgfkeys` error from an unbraced title containing a comma, and a blank page caused by an `\included` file's trailing page break colliding with `\printglossaries`'s own chapter-start page break. Human review and corrections: Brahmi Dalvi reviewed every AWS CLI command and SSH session output in real time before the next step was taken, chose to keep the self-signed TLS certificate rather than pursue a trusted one, and confirmed each AWS resource (the EC2 instance, the opened ports) before it was created. Unresolved assumptions: none remaining for this chapter; the due date was confirmed as October 6, 2026.

Glossary

automation A repeatable, version-controlled procedure executed by software rather than undocumented manual steps. 40

container An isolated, runnable instance of a container image that packages an application with its dependencies. 40

container image An immutable, layered template built from a Dockerfile from which containers are started. 40

deployment The controlled delivery of a versioned change to an environment. 40

evidence A retained record of an actual execution, observation, or review. 40

Kanban board A visual workflow board with columns such as To Do, In Progress, and Complete, used to make work and work-in-progress visible. 40

observability The ability to infer system behavior from signals such as logs, metrics, traces, and health checks. 40

pipeline An automated sequence that builds, tests, packages, and/or deploys a change. 40

rollback A controlled return to a known acceptable version or state after a failed or harmful change. 40

runbook A documented operational procedure for a repeatable task or incident response. 40

security group A stateful virtual firewall attached to an AWS instance that lists the inbound and outbound traffic it allows. 4, 40

synonyms Equivalent terms used in this assignment; record preferred terms to avoid ambiguity in instructions and evidence. 40

TeX Live A cross-platform distribution of TeX, LaTeX, and their packages; Overleaf compiles projects with it. 18, 40

user data A script supplied when an EC2 instance is launched that runs once as root at first boot. 4, 5, 40

Acronyms

AI artificial intelligence. 40

AMI Amazon Machine Image. 4–6, 40

AWS Amazon Web Services. 1, 23, 40

CD continuous delivery or continuous deployment. 31, 40

CI continuous integration. 40

DORA DevOps Research and Assessment. 26, 40

EC2 Amazon Elastic Compute Cloud. 3, 4, 40

IaC infrastructure as code. 40

SLO service-level objective. 40

SWOT strengths, weaknesses, opportunities, and threats. 23, 40

UML Unified Modeling Language. 40

Index

AI agent, 35
    chapter file convention, 35
    DevOps workflow, 35
AI prompt log, 39
Appendix
    AI Agent Usage Guide, 35
    Document Authoring Guide, 31
AWS, 3

dashboard, 26
DevOps, 1
DevSecOps, 23
Docker
    cloud deployment, 3
    LaTeX build, 32
document conventions, 31
document history, v

EC2, 3

introduction, 1

JavaScript
    classes, 12

LaTeX, 31
    Docker, 18

macOS, 32
MacTeX, 32
MiKTeX, 33
Muresan, Darian, iii

Overleaf, 18

project proposal, 23

SWOT, 23
synonyms, 31

template attribution, iii
TeX Live, 18

UML, 12

Windows, 33